Q4 Problems — Answers, Explained

All 9 Q4 problems with code + plain walkthroughs. **Part 1 (dict/set) is your realistic Q4 chance — study these.** Part 2 (window / pointers / prefix) is the hard stuff — read it for awareness, but don't stress if it doesn't fully land.

PART 1 — dictionaries & sets (DO THESE)

most_frequent — the value appearing most often

`[3,1,3,2,3,1]` → 3

```
def most_frequent(nums):
    counts = {}
    for x in nums:
        counts[x] = counts.get(x, 0) + 1     # D1: count each
    best_key = None
    best_count = 0
    for key, value in counts.items():        # D2: find the biggest
        if value > best_count:
            best_count = value
            best_key = key
    return best_key
```

Idea: count each value, then scan for the highest count (tracking key AND value). Short version: `return Counter(nums).most_common(1)[0][0]`

contains_close_duplicates — a repeat within k positions?

`([1,2,3,1], 3)` → True · `([1,2,3,1], 2)` → False

```
def contains_close_duplicates(nums, k):
    last_index = {}                          # value → the last pos
    for i, x in enumerate(nums):
        if x in last_index and i - last_index[x] <= k:
            return True                      # seen recently enough
        last_index[x] = i                    # remember/refresh its
    return False
```

Idea: store *where* you last saw each value. When you meet it again, check if the gap is small enough. `enumerate` gives you position `i` and value `x` together. Walkthrough `([1,2,3,1], 3)`: see 1 at 0, 2 at 1, 3 at 2, then 1 again at 3 → gap 3-0=3 ≤ 3 → True.

common_elements — values in BOTH lists, sorted, no duplicates

`([1,2,2,3], [2,3,4])` → [2,3]

```
def common_elements(a, b):
    return sorted(set(a) & set(b))          # D6: & = intersection
```

Idea: `set(a) & set(b)` gives values in both (and removes duplicates); `sorted` makes it a sorted list. This is `in_both` from your drills.

can_form_note — can note be built from magazine's letters?

`("aab","baa")` → True · `("aab","ab")` → False

```
from collections import Counter
def can_form_note(note, magazine):
    need = Counter(note)                    # how many of each let
    have = Counter(magazine)                # how many I HAVE
    for letter in need:
        if have[letter] < need[letter]:     # not enough of this l
            return False
    return True
```

Idea: count what you need, count what you have, check every needed letter is covered. `("aab","ab")`: need {a:2,b:1}, have {a:1,b:1} → only 1 'a' but need 2 → False.

first_duplicate — first value that repeats

`[2,1,3,5,3,2]` → 3

```
def first_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:                          # D3: SEARCH — return
            return x
        seen.add(x)
    return -1
```

Idea: remember everything you pass; the first value already in `seen` is the answer. This is `first_repeated` from your drills.

PART 2 — patterns (HARD — read for awareness only)

max_subarray_sum_k — biggest sum of any window of size k

`([2,1,5,1,3,2], 3)` → 9 (5+1+3)

```
def max_subarray_sum_k(nums, k):
    window = sum(nums[:k])                     # sum of the FIRST k
    best = window
    for i in range(k, len(nums)):
        window += nums[i] - nums[i-k]          # slide: add new, dro
        best = max(best, window)
    return best
```

Idea (sliding window): don't re-add k numbers each step. Keep a running window sum — when it slides right, add the entering number and subtract the leaving one. Fast.

smallest_difference — closest pair between two lists

`([1,3,15,11,2],[23,127,235,19,8])` → 3 (11 vs 8)

```
def smallest_difference(a, b):
    a, b = sorted(a), sorted(b)              # sort both first
    i = j = 0
    best = abs(a[0] - b[0])
    while i < len(a) and j < len(b):
        best = min(best, abs(a[i] - b[j]))
        if a[i] < b[j]:                      # move the pointer at
            i += 1
        else:
            j += 1
    return best
```

Idea (two pointers): sort both, walk one finger up each. Always advance the finger on the smaller value — that's the only move that could shrink the gap. Beats checking every pair.

count_pairs_with_sum — how many pairs add to target

`([1,5,7,-1,5], 6)` → 3

```
from collections import Counter
def count_pairs_with_sum(nums, target):
    seen = Counter()
    count = 0
    for x in nums:
        count += seen[target - x]            # every earlier partn
        seen[x] += 1
    return count
```

Idea (Two Sum + counting): for each number, its partner is `target - x`. Count how many partners you've already passed, then record this number. `seen[missing]` is 0, no crash.

equal_split_index — index where left sum == right sum

`[1,7,3,6,5,6]` → 3

```
def equal_split_index(nums):
    total = sum(nums)                          # everything
    left = 0                                   # running sum to the l
    for i, x in enumerate(nums):
        right = total - left - x               # everything except le
        if left == right:
            return i
        left += x                              # now include x in the
    return -1
```

Idea (prefix sum): the right side = total − left − current. Sweep once, keeping the left running sum. No nested loop needed.

Bottom line for tomorrow

- **Part 1 problems can genuinely be your Q4.** Counting, sets, first-duplicate — all yours.
- **Part 2 is the hard Q4.** If tomorrow's Q4 is a sliding-window/prefix problem, attempt it last, write brute force, take partial marks. Don't let it eat time from Q1–Q3.
- Target is **350**. Q1 + Q2 + Q3 gets you there. Everything here is a bonus.